

Data-Efficient Adaptation of a Driving VLA to Class 8 Trucks

Satyajeet Das¹, Aaron Buxbaum², Niels Joubert², and Gaurav S. Sukhatme¹

Abstract—Class 8 trucks differ from passenger cars in geometry, dynamics, and maneuvering requirements. As a result, vision-language-action (VLA) models trained for passenger vehicles do not readily transfer to Class 8 trucks, particularly in unstructured scenarios such as accident scenes and construction zones. Rather than training a truck-driving VLA from scratch, we propose an *adapt-then-steer* strategy that adapts an off-the-shelf VLA to generate trajectories for Class-8 trucks in these challenging scenarios. In the *adapt* stage, we use NVIDIA’s *Alpamayo 1.5* as the base model, fine-tuning only its action-generation stack on a few hundred real-world construction and accident-related highway scenarios. In the *steer* stage, we introduce *Flow Velocity Steering* (FVS) to further refine the model’s predictions while holding the adapted VLA fixed. FVS is a compact, flow-time-conditioned residual module that adds learned corrections to the action-space flow velocity used to update the action sequence at each generation step. In open-loop evaluation on a scenario-disjoint held-out set, targeted fine-tuning more than halves single-candidate average displacement error (ADE) and final displacement error (FDE) over the entire 6.4s horizon compared to the base model. Using the same targeted demonstrations, FVS further reduces the fine-tuned model’s full-horizon ADE and FDE by 13.9% and 16.5%, respectively. At matched data budgets, targeted supervision yields 19-26% lower full-horizon ADE than general truck-driving supervision, while the targeted model remains competitive with a model fine-tuned on approximately $65\times$ as many general truck-driving scenarios. These results support *adapt-then-steer* for data-efficient vehicle-domain transfer to Class 8 trucks. Our project website is available at <https://truckvla.github.io>.

I. INTRODUCTION

Vision-language-action (VLA) models for autonomous driving combine multimodal scene representations with trajectory generation, offering a promising starting point for handling complex driving scenarios [1, 2]. However, the trajectory appropriate for a scene depends on the vehicle’s physical embodiment. A Class 8 truck differs from a passenger car in sensor viewpoint, articulated geometry, kinematics, vehicle dynamics, and maneuver timing [3]–[5]. Figure 1(a)–(b) illustrates one such geometric consequence: under the same reference path, the passenger car clears a work zone, whereas trailer off-tracking causes the truck’s swept envelope to enter it. More broadly, one vehicle’s trajectory may not satisfy the geometric and maneuverability requirements of another.

These constraints become especially critical in construction zones and accident-related highway disruptions, the two

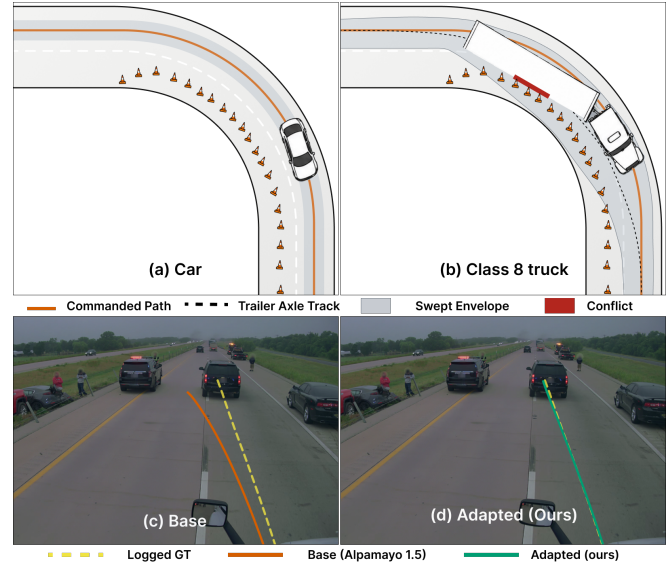


Fig. 1: **Motivation and qualitative example of truck-domain adaptation.** (a)–(b) Under the same reference path, a passenger car clears the work-zone closure, whereas trailer off-tracking causes the Class 8 truck’s swept envelope to enter the closure. (c)–(d) In a held-out accident-related event, off-the-shelf Alpamayo 1.5 deviates from the logged expert truck trajectory, whereas the adapted model follows it more closely. *Adapted* denotes Targeted Truck-SFT followed by FVS. All displayed trajectories in (c)–(d) span 6.4s. The schematics in (a)–(b) are illustrative and not to scale.

target long-tail scenario families studied in this paper. Temporary lane closures and obstructions in these settings can require trajectories that depart from nominal lane following while respecting truck-specific geometric and dynamic constraints. To study transfer to this setting, we use NVIDIA’s Alpamayo 1.5 [2, 6], an open-weight driving VLA that generates continuous trajectories from multimodal driving observations. We evaluate it as an open-loop candidate-trajectory generator intended to provide proposals for downstream planning and safety validation, or to support remote-assist personnel. However, its off-the-shelf predictions deviate substantially from logged expert truck trajectories in these long-tail settings (Fig. 1(c)–(d)).

To our knowledge, data-efficient adaptation of pretrained driving VLAs to trucks in long-tail highway scenarios has not been systematically studied. To address this gap, we leverage Alpamayo’s modular architecture, which consists of an 8.2B-parameter Cosmos-Reason2 vision-language backbone and a 2.3B-parameter flow-matching Action Expert. Its large-scale pretraining motivates reusing the vision-language backbone, while the separate flow-matching Action Expert provides a natural point for targeted refinement [2, 6]. We keep the

¹Satyajeet Das and Gaurav S. Sukhatme are with the Department of Computer Science, University of Southern California, Los Angeles, CA 90089, USA. Emails: {satyajeet, gaurav}@usc.edu.

²Aaron Buxbaum and Niels Joubert are with Stack AV, Pittsburgh, PA, USA. Emails: {abuxbaum, njoubert}@stackav.com.

backbone frozen throughout to isolate how much of the observed prediction gap can be addressed through action-side adaptation alone. This setup motivates our central question: *can limited targeted demonstrations adapt a driving VLA to better match expert truck trajectories in these settings without retraining its vision-language backbone?*

To answer this question, we propose an *adapt-then-steer* strategy. In Stage I, the *adapt* stage, we fine-tune only the action-generation stack on real-world truck scenarios from the target families, yielding *Targeted Truck-SFT*. Stage II, the *steer* stage, asks whether the remaining trajectory error can be further reduced while keeping the adapted VLA fixed. During generation, the Action Expert constructs an action sequence through iterative updates governed by a predicted action-space flow velocity, which describes how the action sequence evolves and is distinct from the truck’s physical velocity [2, 7]. This iterative generation process offers a way to refine predictions without further updating the adapted Action Expert: corrections to an intermediate action sequence can influence its subsequent predictions. To this end, we introduce *Flow Velocity Steering* (FVS), a compact, flow-time-conditioned residual module that adds a learned correction to this velocity at each generation step. The corrected intermediate sequence is passed back to the frozen Action Expert, allowing earlier corrections to influence subsequent action generation. We train FVS through steered rollouts using the same targeted demonstrations as Stage I, exposing it to intermediate action sequences induced by its own earlier corrections.

We evaluate this strategy on a scenario-disjoint test set of construction and accident-related scenarios. For single-candidate prediction, Targeted Truck-SFT reduces average displacement error (ADE) and final displacement error (FDE) over the full 6.4s horizon by approximately 56% relative to the off-the-shelf VLA. Using the same targeted demonstrations, FVS further reduces Targeted Truck-SFT’s full-horizon ADE and FDE by 13.9% and 16.5%, respectively. Crucially, our Stage I comparisons show that targeted adaptation is highly data-efficient, an important benefit given the scarcity of curated long-tail demonstrations. At matched data budgets, targeted supervision yields 19-26% lower full-horizon ADE than general truck-driving supervision. Targeted Truck-SFT also remains competitive with a version of the same base VLA fine-tuned on approximately $65\times$ as many general truck-driving scenarios, which were not specifically selected from the two target long-tail families. Together, these results support our *adapt-then-steer* strategy for vehicle-domain transfer: preserve the pretrained vision-language backbone, target action generation with scenario-aligned demonstrations, and refine trajectories by steering the generative rollout procedure.

Our contributions are the following:

- We demonstrate that, with the vision-language backbone frozen, fine-tuning only the action-generation stack on targeted truck demonstrations reduces full-horizon trajectory error by approximately 56% relative to the off-the-shelf VLA.

- We introduce FVS, a compact, flow-time-conditioned residual module that adds learned corrections to the action-space flow velocity at each generation step. FVS complements the fine-tuning process by further reducing full-horizon ADE and FDE by 13.9% and 16.5%, respectively, while keeping the adapted VLA fixed.
- We show that targeted supervision yields lower full-horizon trajectory error at matched scenario counts than general truck-driving supervision, while the targeted model remains competitive with one fine-tuned on approximately $65\times$ as many general scenarios.

II. RELATED WORK

a) Autonomous-driving VLAs and long-tail adaptation.: Recent autonomous-driving VLAs extend multimodal scene understanding to trajectory generation. DriveVLM uses vision-language reasoning for scene analysis and hierarchical planning, while EMMA predicts planner trajectories and other driving outputs within a unified multimodal model [1, 8]. Alpamayo-R1 combines Chain-of-Causation reasoning with a generative trajectory predictor for long-tail driving situations [2]. Recent work explores several interfaces for adapting driving policies to challenging scenarios. SteerVLA learns a control-aligned language interface through which a high-level vision-language model guides a low-level driving policy [9]; CLAP optimizes soft prompts for a frozen policy while preserving nominal behavior [10]; and TakeVLA uses takeover data, language supervision, and reinforcement post-training [11]. More generally, adapters and low-rank updates provide parameter-efficient interfaces for specializing large pretrained backbones [12, 13]. Our work focuses on action-side adaptation for vehicle-domain transfer: we specialize the action-generation stack using real truck trajectories while preserving the pretrained vision-language backbone.

b) Autonomous trucking and vehicle-domain transfer.: Truck autonomy introduces sensing, embodiment, and operating-domain differences that are not fully represented by passenger-vehicle benchmarks. MAN TruckScenes documents truck-specific challenges including elevated sensor perspectives, trailer occlusions, and terminal environments [3], while TruckDrive emphasizes the long sensing and anticipation ranges required for highway-speed trucking [4]. MVAdapt explicitly formulates a vehicle-domain gap and conditions features from a frozen scene encoder on vehicle properties to transfer an end-to-end policy across vehicle embodiments in simulation [5]. Our work focuses on this domain gap by adapting an open-weight autonomous-driving VLA to Class 8 trucks using real-world trajectories from construction and accident-related scenarios.

c) Residual adaptation of generative action policies.: Residual policy learning augments a nominal controller or policy with a learned correction and has been used to improve otherwise imperfect robot policies [14, 15]. Recent work adapts pretrained generative policies by modifying their generated actions or the action-generation process while keeping the original policy weights fixed. FlowCorrect uses

gated low-rank adapters to correct intermediate flow velocities from sparse human interventions, training on remaining-flow-time velocity targets [16]. RFS combines residual action adaptation with reinforcement learning over the initial latent distribution [17]; GLOVES refines completed action chunks using a separately learned expert flow [18]; and RL²-VLA learns an offline-RL policy conditioned on Action Expert latents and composes its flow velocity with that of a frozen VLA [19]. Building on the broader residual learning paradigm, FVS corrects intermediate action-space flow velocities using a separate, lightweight residual network. We train FVS offline from logged expert truck demonstrations using intermediate action-space bridge supervision and rollout-level imitation supervision, exposing it to generative action states induced by its own earlier corrections while keeping the fine-tuned policy fixed.

III. METHOD

A. Problem Formulation and Overview

We adapt a pretrained driving VLA model to generate candidate trajectories for a Class 8 truck platform. Given an observation $o = (\mathcal{I}, \mathcal{H}, q)$, where $\mathcal{I}$ denotes the synchronized multi-camera observation history, $\mathcal{H}$ the recent ego-motion history, and q the trajectory generation prompt, the target $Y^* = \{(p_i^*, R_i^*)\}_{i=1}^N$ is the logged future pose trajectory of the truck's vehicle control point (VCP), where p_i^* and R_i^* denote its future position and orientation. The VCP serves as the common reference point for both logged and predicted truck motion, and each target trajectory is expressed in the coordinate frame of the current VCP pose.

Following Alpamayo 1.5 [2, 6], we retain its native normalized acceleration-curvature action representation and associated fixed conversion routines. The mapping $\mathcal{T}$ constructs action targets from logged trajectories, while $\mathcal{D}$ denormalizes and integrates generated actions using the native unicycle model:

$$\begin{aligned} x^* &= \mathcal{T}(Y^*, \mathcal{H}) \in \mathbb{R}^{N \times 2}, \\ \hat{Y} &= \mathcal{D}(x_K, \mathcal{H}), \end{aligned} \quad (1)$$

where x^* is the expert truck action sequence and x_K is the generated action after K flow-integration steps. The fixed decoder is differentiable with respect to the generated actions during FVS training. Trajectory-generation settings are given in Sec. IV-A.

Our adaptation proceeds in two stages (Fig. 2). Stage I fine-tunes the action-generation stack with the vision-language backbone frozen. Stage II freezes the adapted policy and learns FVS to correct its action-space flow velocity throughout generation.

B. Truck-Specific Action-Expert Fine-Tuning

Let F_ψ denote the pretrained vision-language backbone and $z = F_\psi(o)$ its multimodal context. During SFT, the backbone parameters ψ remain frozen, while the flow-matching action stack, parameterized by ϕ and comprising the Action Expert and its action input/output projections, is fine-tuned on the truck demonstrations.

We retain Alpamayo 1.5's native conditional flow-matching objective [2, 6, 7]. For an expert truck action sequence x^* , we sample Gaussian noise $\varepsilon \sim \mathcal{N}(0, I)$ and generative flow time $\tau \in [0, 1]$ using the native SFT sampling distribution. We construct $x_\tau = (1 - \tau)\varepsilon + \tau x^*$ and optimize ϕ using

$$\mathcal{L}_{\text{SFT}} = \mathbb{E} \left[\|v_\phi(x_\tau, \tau, z) - (x^* - \varepsilon)\|_F^2 \right], \quad (2)$$

where v_ϕ is the predicted action-space flow velocity. At inference, sampling starts from $x_0 \sim \mathcal{N}(0, I)$ and follows the native Euler solver with step sizes $\Delta\tau_k = \tau_{k+1} - \tau_k$:

$$x_{k+1} = x_k + \Delta\tau_k v_\phi(x_k, \tau_k, z), \quad k = 0, \dots, K-1. \quad (3)$$

During sampling, the context z remains fixed while x_k evolves in generative flow time τ . Flow velocity describes the rate of change of normalized actions with respect to τ , not the truck's physical velocity.

C. Flow Velocity Steering

FVS acts during the adapted policy's iterative action-generation process rather than correcting only the completed action sequence. While the native SFT objective in Eq. (2) fits action-space flow velocities, it does not explicitly supervise the trajectory obtained after a complete generative rollout and decoding. This motivates a complementary refinement stage that directly supervises the generated trajectory while updating only a compact residual module. To address this, we freeze the adapted policy after SFT and introduce FVS as a trainable residual module conditioned on flow time. This allows us to supervise both intermediate corrections and the final decoded trajectory through complete generative rollouts.

Specifically, at each Euler step k , the frozen Action Expert is evaluated at the current steered action state x_k , returning the nominal action-space flow velocity $v_k^{\text{SFT}} = v_\phi(x_k, \tau_k, z)$ and its hidden action representation h_k . FVS then predicts a residual correction to this velocity:

$$\Delta v_{\theta,k} = A_\theta(x_k, v_k^{\text{SFT}}, h_k, \tau_k, k). \quad (4)$$

The steered sampler applies this correction to update the action state:

$$x_{k+1} = x_k + \Delta\tau_k (v_k^{\text{SFT}} + \alpha \Delta v_{\theta,k}), \quad (5)$$

where α controls the steering strength; $\alpha = 0$ recovers the frozen SFT sampler. Because the resulting intermediate action sequence is passed back to the frozen Action Expert at the next Euler step, FVS dynamically steers the entire rollout.

D. Trajectory-Aware Steering Objective

We train FVS from the same logged expert demonstrations used for SFT, using action-space bridge supervision at intermediate flow steps and imitation supervision on complete steered rollouts. For each training example, the frozen SFT and steered rollouts share the same initial Gaussian noise. The matched frozen rollout provides a training-only difficulty

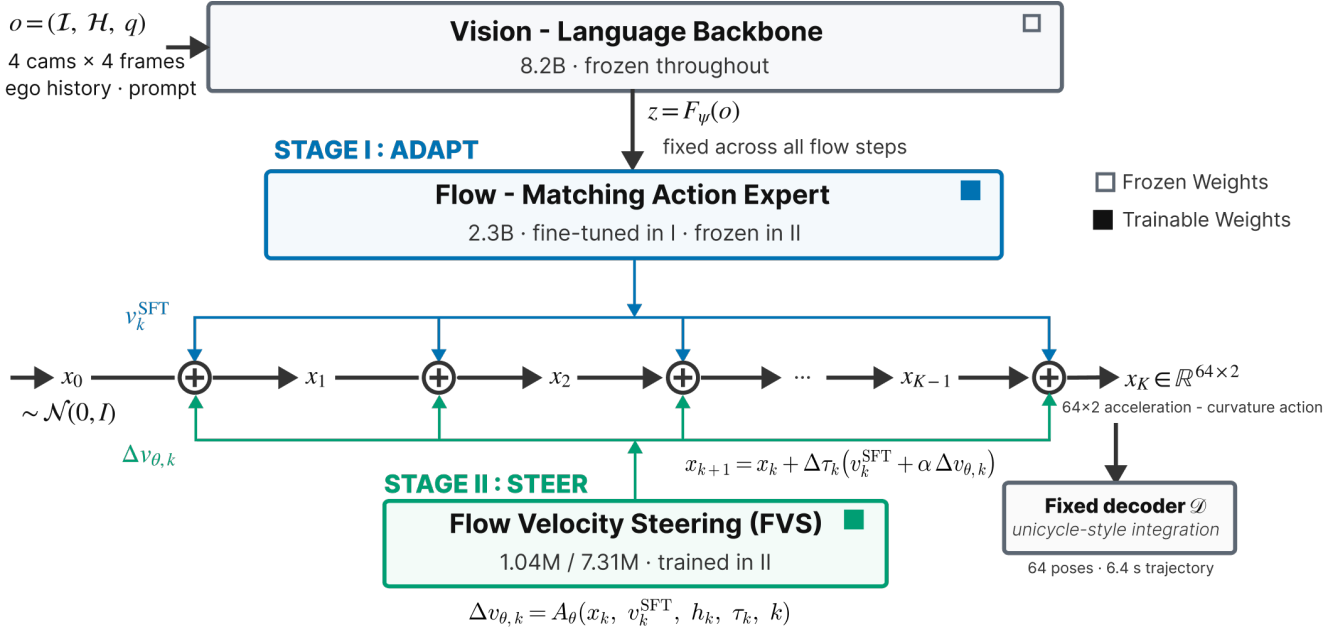


Fig. 2: Overview of the proposed adapt-then-steer pipeline. Stage I fine-tunes the action-generation stack with the vision-language backbone frozen; Stage II freezes the adapted policy and trains FVS to correct its action-space flow velocity throughout the Euler rollout before fixed trajectory decoding.

weight $\eta \in [0, 1]$, derived from its trajectory error using training-set statistics; larger errors yield higher weights.

We construct a bridge target to guide the current steered action sequence toward the expert action. Let $R_k = \sum_{j=k}^{K-1} \Delta \tau_j$ denote the remaining flow time. Over this interval, the constant action-space flow velocity $(x^* - x_k)/R_k$ would move the current action state x_k to the expert action x^* . Since FVS acts as a residual on the frozen Action Expert’s action-space flow velocity, we define the bridge target

$$\Delta v_k^* = \eta \beta \left(\frac{x^* - x_k}{\max(R_k, R_{\min})} - v_k^{\text{SFT}} \right), \quad (6)$$

where $R_{\min}$ stabilizes the denominator and β controls the target scale. The bridge loss $\mathcal{L}_{\text{bridge}}$ penalizes deviation of the predicted correction $\Delta v_{\theta,k}$ from the bridge target Δv_k^* .

After the complete steered rollout, we decode $\hat{Y}_\theta = \mathcal{D}(x_K^\theta, \mathcal{H})$ and compute a per-example imitation loss E_θ supervising the generated action sequence and decoded trajectory against the logged expert demonstration. Let E_{SFT} denote the same loss evaluated on the matched-noise frozen SFT prediction. We use $\mathcal{L}_{\text{traj}} = E_\theta$ and penalize regression relative to the frozen policy using $\mathcal{L}_{\text{nr}} = [E_\theta - E_{\text{SFT}}]_+$, where $[a]_+ = \max(a, 0)$.

The complete objective is

$$\mathcal{L}_{\text{FVS}} = \lambda_b \mathcal{L}_{\text{bridge}} + \lambda_t \mathcal{L}_{\text{traj}} + \lambda_{\text{nr}} \mathcal{L}_{\text{nr}} + \mathcal{L}_{\text{reg}}, \quad (7)$$

where $\mathcal{L}_{\text{reg}}$ collects correction-magnitude and architecture-specific regularization terms.

a) Rollout-aware optimization: We optimize FVS through complete steered rollouts under each fixed logged observation. Because each correction modifies the action state x_k , earlier FVS outputs affect the states encountered at later flow steps. FVS is therefore trained on action

states induced by its own corrections rather than only on frozen-policy states, reducing the resulting state-distribution mismatch [20]. The adapted VLA remains fixed throughout, and training requires neither environment interaction nor additional expert queries.

IV. EXPERIMENTAL RESULTS

A. Dataset and Evaluation Protocol

a) Datasets and splits: We study two target long-tail scenario families: construction zones and accident-related road disruptions. Each targeted scenario records an expert truck driver’s encounter with one of these situations, including the maneuver through it. Multiple prediction windows are sampled within each scenario. The targeted training set contains 229 scenarios, comprising 138 accident-related and 91 construction scenarios. The general training set contains 14,891 truck-driving scenarios not specifically selected for these families. Both corpora contain real truck-driving data. All primary methods are evaluated on the same scenario-disjoint held-out set of 26 real-world truck encounters spanning construction zones and accident-related road disruptions. The test scenarios are disjoint from both the targeted and general training corpora, and checkpoints and hyperparameters were selected without reference to them.

b) Compared methods: *Base* denotes the off-the-shelf Alpmayo 1.5 checkpoint. *Targeted Truck-SFT* and *General Truck-SFT* fine-tune the action-generation stack on the targeted and general training sets, respectively, while preserving the vision-language backbone. *General* $\rightarrow$ *Targeted SFT* initializes from General Truck-SFT and is subsequently fine-tuned on the targeted scenarios.

For post-SFT refinement, FVS and the *final-action residual* baseline both start from the same frozen Targeted Truck-

Method	$K_{\text{cand}} = 1$: single candidate				$K_{\text{cand}} = 16$: oracle best-of-16			
	ADE@1s	ADE@3s	ADE@6.4s	FDE@6.4s	minADE@1s	minADE@3s	minADE@6.4s	minFDE@6.4s
CTRV	0.1643	0.7003	2.0430	5.8412	—	—	—	—
Base	0.1530 ± 0.0018	0.6836 ± 0.0440	2.6358 ± 0.1493	7.1830 ± 0.2868	0.0621 ± 0.0001	0.2556 ± 0.0082	0.9402 ± 0.0038	2.3849 ± 0.0118
Targeted Truck-SFT	0.0600 ± 0.0008	0.3120 ± 0.0081	1.1650 ± 0.0184	3.1590 ± 0.0213	0.0413 ± 0.0004	0.1855 ± 0.0029	0.6968 ± 0.0115	1.8118 ± 0.0398
General Truck-SFT	0.0584 ± 0.0017	0.3146 ± 0.0194	1.2868 ± 0.0049	3.7404 ± 0.0824	0.0361 ± 0.0001	0.1659 ± 0.0051	0.6634 ± 0.0044	1.7652 ± 0.0240
General→Targeted SFT	0.0580 ± 0.0017	0.3121 ± 0.0168	1.2539 ± 0.0422	3.6012 ± 0.0703	0.0371 ± 0.0008	0.1716 ± 0.0045	0.6697 ± 0.0009	1.7587 ± 0.0313
Final-Action Residual (MLP)	0.0595 ± 0.0006	0.3060 ± 0.0043	1.1362 ± 0.0092	3.1071 ± 0.0186	0.0402 ± 0.0003	0.1813 ± 0.0012	0.6918 ± 0.0073	1.8001 ± 0.0232
Final-Action Residual (Transformer)	0.0589 ± 0.0006	0.3010 ± 0.0032	1.1276 ± 0.0081	3.0002 ± 0.0175	0.0391 ± 0.0003	0.1801 ± 0.0016	0.6876 ± 0.0051	1.7992 ± 0.0210
FVS (MLP)	0.0481 ± 0.0003	0.2659 ± 0.0004	1.0065 ± 0.0035	2.6527 ± 0.0171	0.0363 ± 0.0000	0.1658 ± 0.0001	0.6613 ± 0.0027	1.7419 ± 0.0115
FVS (Transformer)	0.0468 ± 0.0003	0.2656 ± 0.0009	1.0035 ± 0.0035	2.6383 ± 0.0156	0.0354 ± 0.0000	0.1642 ± 0.0006	0.6573 ± 0.0012	1.7003 ± 0.0102

TABLE I: Trajectory accuracy on the 26-scenario held-out truck long-tail test set. $K_{\text{cand}} = 1$ evaluates one candidate, whereas $K_{\text{cand}} = 16$ reports oracle best-of-16 minADE/minFDE. Learned-model values are mean $\pm$ sample standard deviation across inference-noise seeds. CTRV is deterministic and reported only for $K_{\text{cand}} = 1$. Errors are in metres; lower is better. Bold indicates the lowest mean in each column. Final-action residual and FVS models use the same frozen Targeted Truck-SFT checkpoint.

SFT checkpoint. The final-action residual applies a learned correction to the completed acceleration-curvature sequence before trajectory decoding, whereas FVS corrects the action-space flow velocity at every Euler step during generation. We instantiate both approaches with MLP and Transformer residual networks. Within each architecture, the two approaches use the same targeted training set, optimizer settings, training-update budget, and trainable-parameter count. The MLP and Transformer residual modules contain 1.044M and 7.308M trainable parameters, respectively. Each FVS module therefore adds less than 0.1% to the parameter count of the complete frozen VLA.

As a deterministic motion-only reference, we include a constant-turn-rate-and-velocity (CTRV) baseline [21]. CTRV uses the full causal ego-motion history also supplied to Base. For each adjacent pose pair, we estimate signed longitudinal speed by projecting the displacement onto the midpoint heading and estimate yaw rate from the wrapped heading difference, in both cases using the actual time interval. We then hold the median speed and median yaw rate over the full history constant and analytically propagate a trajectory from the current pose, using the same prediction horizon and sampling interval as the learned models.

c) Data-efficiency protocol: For each evaluated training-set budget, scenarios are randomly sampled from the targeted and general corpora. We train three independently sampled subset runs for each supervision source at each budget. All subset runs, together with the full-data Targeted Truck-SFT and General Truck-SFT runs, use the same training-update budget. The selected checkpoint from each run is evaluated on the common held-out test set. Figure 3 reports the mean and standard deviation across the three subset runs.

d) Trajectory generation: Each prediction uses four synchronized truck-mounted camera streams with four frames per camera, preserving Alpayo 1.5’s default four-camera, four-frame-per-camera input count. The truck rig provides corresponding camera views, with mounting positions adapted to the Class 8 vehicle embodiment. Each learned candidate is generated as a sequence of 64 pairs of normalized longitudinal acceleration and curvature using $K = 10$ Euler flow-integration steps. The fixed Alpayo decoder maps this sequence to a 6.4s VCP trajectory containing

64 future poses at 0.1s intervals. Candidate generation is invoked at 1Hz along each scenario, so successive prediction horizons overlap. All evaluations are open-loop: each prediction is conditioned on the logged camera observations and ego-motion history at its timestamp, and is scored against the logged expert trajectory over the same horizon.

e) Metrics: We denote the number of independently sampled candidate trajectories by K_{cand} , distinct from the number of flow-integration steps K . For $K_{\text{cand}} = 1$, ADE at horizon H is the mean Euclidean VCP-position error over the predicted poses through H , whereas FDE is the VCP-position error at that horizon. We report ADE at 1, 3, and 6.4s, and FDE at 6.4s. At 6.4s, we additionally report lateral and longitudinal ADE, defined as mean absolute VCP-position errors along the corresponding coordinate axes, and final heading error, defined as the absolute angular difference between the predicted and logged final orientations.

For $K_{\text{cand}} = 16$, we report oracle best-of-16 minADE and minFDE. For each prediction window, each metric independently selects its lowest-error candidate before the per-window minima are averaged. The selected candidate may therefore differ across metrics. These oracle quantities measure best-of-set agreement with the logged trajectory and do not assume an online selector with access to ground truth.

For both candidate settings, metrics are averaged over all held-out prediction windows. Each scenario contributes the same number of windows and therefore has equal weight in the aggregate. Because each window is evaluated against one logged expert trajectory, alternative valid maneuvers may incur nonzero displacement error. For each learned-model checkpoint and inference-noise seed, metrics are first aggregated over the common set of evaluation windows. Tables I and II report the mean and sample standard deviation of these per-seed aggregates. The reported deviations therefore characterize inference-sampling variability for fixed checkpoints and evaluation scenarios.

B. Targeted Fine-Tuning for Truck Adaptation

We compare Base with Targeted Truck-SFT trained on 229 targeted scenarios (Table I). With the vision-language backbone frozen, action-side fine-tuning reduces full-horizon single-candidate ADE and FDE by approximately 56%, with ADE decreasing from 2.636 to 1.165m. The CTRV baseline

Method	Lateral ADE ↓	Longitudinal ADE ↓	Heading Error ↓
CTRV	0.7958	1.6934	1.6162
Base	1.0204 ± 0.0638	2.1706 ± 0.2306	3.2926 ± 0.8549
Targeted Truck-SFT	0.4749 ± 0.0130	0.9323 ± 0.0197	0.8851 ± 0.0232
General Truck-SFT	0.4832 ± 0.0152	1.0929 ± 0.0402	1.1377 ± 0.0459
General→Targeted SFT	0.4932 ± 0.0140	1.0385 ± 0.0298	1.1192 ± 0.0553
Final-Action Residual (MLP)	0.4634 ± 0.0070	0.9115 ± 0.0115	0.8590 ± 0.0172
Final-Action Residual (Transformer)	0.4592 ± 0.0068	0.9009 ± 0.0117	0.8512 ± 0.0171
FVS (MLP)	0.4059 ± 0.0053	0.7738 ± 0.0127	0.7644 ± 0.0120
FVS (Transformer)	0.3870 ± 0.0051	0.7650 ± 0.0110	0.7169 ± 0.0105

TABLE II: Supporting $K_{\text{cand}} = 1$ trajectory-geometry metrics at 6.4s on the held-out test set. Learned-model values are mean $\pm$ sample standard deviation across inference-noise seeds; CTRV is deterministic. Lateral and longitudinal ADE are in metres; final heading error is in degrees. Lower is better; bold indicates the lowest mean in each column.

provides a motion-only reference by extrapolating recent truck motion without visual or language input. Although Base has lower single-candidate ADE than CTRV at 1 and 3s, its full-horizon ADE and FDE exceed those of CTRV, motivating truck-specific adaptation of the off-the-shelf policy. Targeted Truck-SFT achieves lower error than both Base and CTRV at every reported single-candidate horizon.

The improvement is consistent across the held-out set: Targeted Truck-SFT lowers scenario-mean ADE@6.4s relative to Base in all 26 scenarios, with mean reductions of approximately 63% for accident-related scenarios and 52% for construction scenarios. Targeted Truck-SFT also lowers all reported oracle best-of-16 errors relative to Base, improving best-of-set agreement with logged truck trajectories. The gains span multiple components of trajectory geometry: lateral and longitudinal ADE are roughly halved, while final heading error decreases from 3.293° to 0.885° (Table II).

As a parameter-free alternative to fine-tuning, we kept Base fixed and evaluated truck-specific alternatives to the default prompt q (Sec. III-A). These prompts were informed by expert truck-driver feedback on turning behavior, trailer off-tracking, braking, clearance, and control smoothness. In a separate diagnostic on held-out events using one inference-noise seed, none of the tested prompts reduced single-candidate ADE@6.4s relative to the default prompt. Accordingly, all main learned-model comparisons use the default prompt q .

Together, these results show that action-side adaptation substantially improves agreement with logged truck trajectories while keeping the vision-language backbone frozen.

C. Data Efficiency of Targeted Supervision

The gains obtained from 229 targeted scenarios motivate a closer examination of data efficiency. We ask whether targeted long-tail demonstrations provide more effective supervision than the same number of general truck-driving scenarios. Figure 3 compares the two supervision sources at matched training-scenario counts. At the common budgets of 2, 10, 100, and 200 scenarios, targeted training achieves approximately 19-26% lower mean ADE@6.4s than general truck-driving supervision on the held-out long-tail distribu-

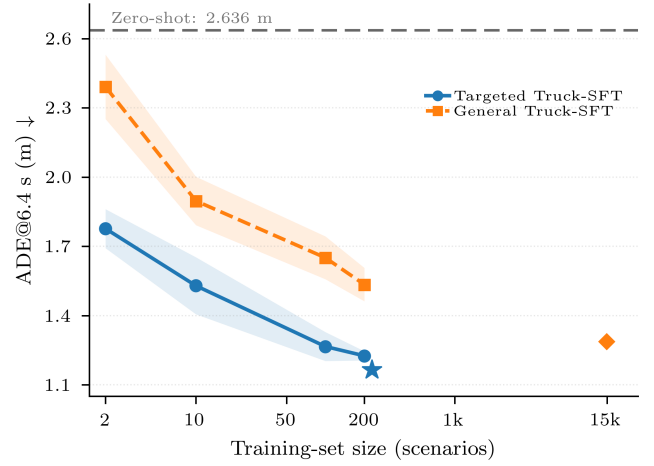


Fig. 3: **Data efficiency of truck adaptation.** ADE@6.4s versus training-set size for targeted long-tail and general truck-driving supervision. Curves show mean performance across three independently trained subset runs at each evaluated size; shaded bands denote ± 1 standard deviation. The blue star and orange diamond denote the full-data Targeted Truck-SFT (229 scenarios) and General Truck-SFT (14,891 scenarios) checkpoints, respectively, shown as separate reference points. The dashed gray line denotes the zero-shot Base model.

tion. For targeted supervision, error decreases with training-set size, with diminishing gains at larger evaluated budgets.

We next compare the full-data checkpoints (Table I). For single-candidate prediction, General Truck-SFT is slightly more accurate at 1s, whereas Targeted Truck-SFT achieves lower full-horizon error: 1.165 versus 1.287m ADE@6.4s, and 3.159 versus 3.740m FDE@6.4s. General→Targeted SFT does not consistently improve over targeted-only adaptation. The ordering differs under oracle best-of-16 evaluation: General Truck-SFT obtains the lowest minADE among the SFT variants, while General→Targeted SFT obtains the lowest minFDE.

Overall, Targeted Truck-SFT trained on only 229 targeted scenarios remains competitive with General Truck-SFT trained on approximately $65\times$ as many general truck-driving scenarios.

D. Post-SFT Refinement with Flow Velocity Steering

We next evaluate whether FVS can further reduce trajectory error while keeping Targeted Truck-SFT fixed (Table I), using the same 229 targeted training scenarios. Relative to Targeted Truck-SFT, Transformer FVS reduces full-horizon ADE and FDE by 13.9% and 16.5%, respectively, compared with 3.2% and 5.0% for the Transformer final-action residual. The FVS gains also extend to shorter horizons. At matched trainable-parameter counts and training-update budgets, FVS outperforms the corresponding final-action residual for both MLP and Transformer architectures.

The additional FVS gain is likewise consistent across the held-out set: Transformer FVS lowers scenario-mean ADE@6.4s relative to Targeted Truck-SFT in all 26 scenarios, with mean reductions of approximately 16% for accident-related scenarios and 11% for construction sce-

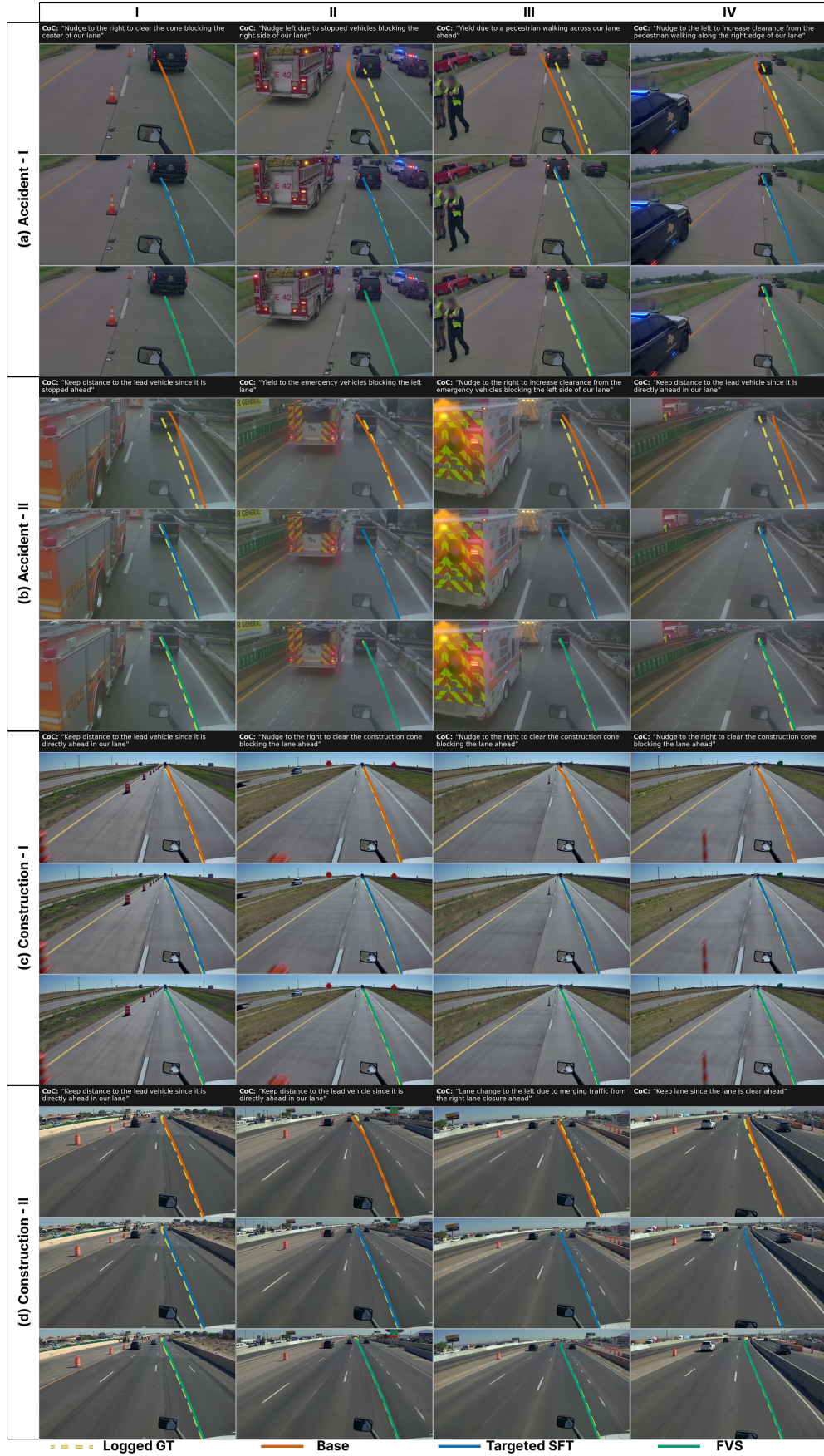


Fig. 4: **Illustrative adapt-then-steer results on held-out truck scenarios.** Columns I–IV progress chronologically, with Base Chain-of-Causation (CoC) annotations. Rows compare Base, Targeted Truck-SFT, and Transformer FVS (top to bottom) against the logged truck trajectory (yellow dashed), using $K_{\text{cand}} = 1$ and a shared inference-noise seed.

narios. Transformer FVS also lowers all reported oracle minADE and minFDE values relative to Targeted Truck-SFT, improving both single-candidate and best-of-16 trajectory accuracy. Table II additionally shows reductions in lateral ADE, longitudinal ADE, and final heading error, indicating improved lateral placement, longitudinal progress, and final orientation. Together, these comparisons support FVS as an effective post-SFT refinement strategy. At inference, FVS requires one lightweight residual-network evaluation per existing Euler step, without increasing the number of flow-integration steps or vision-language backbone passes.

E. Qualitative Analysis

Figure 4 shows selected illustrative timestamps from two accident-related and two construction scenarios, comparing Base, Targeted Truck-SFT, and Transformer FVS against the logged expert truck trajectories. In several displayed accident-related frames, Base’s Chain-of-Causation describes a plausible high-level response, such as yielding or increasing clearance around emergency vehicles and pedestrians, while its generated trajectory differs substantially from the logged truck motion. Targeted Truck-SFT brings the predicted maneuver closer to the demonstrated expert trajectory. The Transformer FVS prediction either remains close to the SFT prediction or shows a smaller correction toward the logged motion. The construction examples show a similar pattern under temporary work-zone constraints, including cone intrusions and lane-closure maneuvers. These examples are consistent with the aggregate quantitative results: targeted SFT provides the primary trajectory correction, while FVS further refines the adapted predictions. Complete temporal sequences are provided in the supplementary videos.

V. CONCLUSION

We studied whether an autonomous-driving VLA can be adapted to Class 8 trucks without retraining its vision-language backbone. With the backbone frozen, fine-tuning the action-generation stack on 229 scenario-aligned truck events reduces full-horizon ADE and FDE by approximately 56%. At matched data budgets, scenario-aligned supervision consistently yields lower full-horizon ADE than general truck-driving supervision, while Targeted Truck-SFT remains competitive with General Truck-SFT trained on approximately $65\times$ as many general truck-driving scenarios. Starting from Targeted Truck-SFT, Flow Velocity Steering (FVS) further reduces full-horizon ADE and FDE by 13.9% and 16.5%, respectively, while keeping the adapted VLA fixed. Together, these results support an *adapt-then-steer* strategy for vehicle-domain transfer to Class 8 trucks: preserve the pretrained vision-language backbone, specialize action generation with scenario-aligned demonstrations, and refine trajectory generation by steering the adapted policy’s action-space flow velocity. Our evaluation measures open-loop agreement with logged truck trajectories rather than closed-loop driving performance. Future work will extend this study to closed-loop evaluation and simulator-based post-training

in a validated vision-based truck simulator with realistic vehicle dynamics.

REFERENCES

- [1] J.-J. Hwang, R. Xu, H. Lin, W.-C. Hung, J. Ji, K. Choi, D. Huang, T. He, P. Covington, B. Sapp *et al.*, “Emma: End-to-end multimodal model for autonomous driving,” *arXiv preprint arXiv:2410.23262*, 2024.
- [2] Y. Wang, W. Luo, J. Bai, Y. Cao, T. Che, K. Chen, Y. Chen, J. Diamond, Y. Ding, W. Ding *et al.*, “Alpamayo-r1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail,” *arXiv preprint arXiv:2511.00088*, 2025.
- [3] F. Fent, F. Kutteneich, F. Ruch, F. Rizwin, S. Juergens, L. Lechermann, C. Nissler, A. Perl, U. Voll, M. Yan *et al.*, “Man trucksccenes: A multimodal dataset for autonomous trucking in diverse conditions,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 62 062–62 082, 2024.
- [4] F. Ghilotti, E. Palladin, S. Brucker, A. Sigal, M. Bijelic, and F. Heide, “Truckdrive: Long-range autonomous highway driving dataset,” *arXiv preprint arXiv:2603.02413*, 2026.
- [5] H. Oh and J. Park, “Mvadap: Zero-shot multi-vehicle adaptation for end-to-end autonomous driving,” *arXiv preprint arXiv:2604.11854*, 2026.
- [6] NVIDIA, “Alpamayo 1.5 (alpamayo-1.5-10b),” 2026. [Online]. Available: <https://huggingface.co/nvidia/Alpamayo-1.5-10B>
- [7] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” *arXiv preprint arXiv:2210.02747*, 2022.
- [8] X. Tian, J. Gu, B. Li, Y. Liu, Y. Wang, Z. Zhao, K. Zhan, P. Jia, X. Lang, and H. Zhao, “Drivevlm: The convergence of autonomous driving and large vision-language models,” *arXiv preprint arXiv:2402.12289*, 2024.
- [9] T. Gao, C. Tan, C. Glossop, T. Gao, J. Sun, K. Stachowicz, S. Wu, O. Mees, D. Sadigh, S. Levine *et al.*, “Steervla: Steering vision-language-action models in long-tail driving scenarios,” *arXiv preprint arXiv:2602.08440*, 2026.
- [10] R. Zhu, Y. He, B. Zheng, Z. Zhao, A. Chalhoub, Q. Zhang, and Z. M. Mao, “Clap: Contrastive latent-space prompt optimization for end-to-end autonomous driving,” *arXiv preprint arXiv:2605.17284*, 2026.
- [11] Y. Gao, D. Liu, Q. Zhang, Y. Zheng, H. Tian, G. Li, H. Ye, L. Chen, D.-W. Ding, and D. Zhao, “Learning from mistakes: Post-training for driving VLA with takeover data,” *arXiv preprint arXiv:2603.14972*, 2026.
- [12] N. Hounsby, A. Giurghi, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for nlp,” in *International conference on machine learning*. PMLR, 2019, pp. 2790–2799.
- [13] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [14] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, “Residual policy learning,” *arXiv preprint arXiv:1812.06298*, 2018.
- [15] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *2019 international conference on robotics and automation (ICRA)*. IEEE, 2019, pp. 6023–6029.
- [16] E. Welte, Y. Shi, R. Wolf, M. Gilles, and R. Rayyes, “Flowcorrect: Efficient interactive correction of generative flow policies for robotic manipulation,” *arXiv preprint arXiv:2602.22056*, 2026.
- [17] E. Su, T. Westenbroek, A. Nagabandi, and A. Gupta, “Rfs: Reinforcement learning with residual flow steering for dexterous manipulation,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [18] L. Sun, J. Ji, H. Chen, J. Zhou, and M. R. Walter, “Flow-based policy adaptation without policy updates,” *arXiv preprint arXiv:2606.06461*, 2026.
- [19] D. M. S. Tan, S. Shailesh, S. Iyer, W. W. J. Teo, Y. Ju, Q. Gu, and G. Sartoretti, “Rl²-vla: Adaptive rl latent compositional steering with test-time scaling for vision-language-action models,” *arXiv preprint arXiv:2607.26991*, 2026.
- [20] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.

- [21] R. Schubert, E. Richter, and G. Wanielik, "Comparison and evaluation of advanced motion models for vehicle tracking," in *2008 11th international conference on information fusion*. IEEE, 2008, pp. 1–6.